

Optimierung des Entwicklungsworkflows in der Webentwicklung durch Codegenerierung und OpenAPI

Nicolas Ostermann (82796)

Bachelorarbeit
Prof. Dr. Rainer Werthebach
Hochschule Aalen
Heitec AG
01.06.2024

Über Heitec AG

Gegründet 1973 von Richard Heindl als Beratungs- und Konstruktionsfirma, expandierte HEITEC kontinuierlich und ist heute mit über 1.200 Mitarbeitern und einem Umsatz von 140 Millionen Euro im Jahr 2022 ein führendes Technologieunternehmen. Mit Hauptsitz in Erlangen seit 2017, bietet das Unternehmen umfassende Industrielösungen in sechs Hauptabteilungen: Automatisierung, Produktionssysteme, Mess- und Prüftechnik, Elektronik, Softwarelösungen sowie Validierung und Dokumentation.

Inhaltsverzeichnis

01

Einleitung

Problemstellung
Ziel der Arbeit

02

Grundlagen

Basiswissen Technologien
Basiswissen Projektmanagement

03

Problemanalyse

Wirtschaftlicher, menschlicher &
technischer Faktoren

04

Lösungskonzept

Konzept in Bezug auf die
Problemanalyse

05

Umsetzung Lösungskonzept

Implementierung
Inbetriebnahme

06

Abschluss

Evaluation
Zusammenfassung & Ausblick
Demo

Problemstellung

Kundenbedürfnisse und Entwicklungspraktiken:

Anpassung kann oft zu Ineffizienzen führen → insbesondere bei Full-Stack-Entwicklungsansätzen

Boilerplate-Code:

Die Erstellung und Pflege nimmt viel Zeit in Anspruch. Dies beeinträchtigt:

- Entwicklungsgeschwindigkeit
- Code-Konsistenz und Klarheit

Auswirkungen:

- Reduzierte Codequalität
- Beeinträchtigte Wartbarkeit
- Begrenzte Skalierbarkeit

Relevanz von Best Practices:

Durchsetzung von Best Practices ist entscheidend → einheitlichen Code fördern und langfristige Qualität/Erweiterbarkeit gewährleisten.

Abgrenzung:

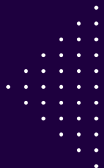
KI generell, kein Bezug zu LLMs und NLP



Ziel der Arbeit

Optimierung des Arbeitsablaufs zwischen Client- und Server-Verbindungen

Def. und Implementierung eines Optimierten Workflows	Klaren Prozess schaffen → Informationsaustausch zwischen Client und Server effizienter
Reduktion Boilerplate-Code	Reduzierter Aufwand für Erstellung von standardisierten Code
Steigerung Effizienz, Qualität und Wartbarkeit	Besserer Workflow und weniger Boilerplate Code soll Effizienz und Qualität verbessern
Sicherstellung der Erweiterbarkeit	Entwickelte Systeme sollen flexibel und anpassbar bleiben



Grundlagen



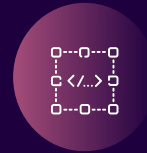
Angular

Googles Frontend
Framework für SPAs



.NET

Microsofts Software
Framework



TypeScript

Microsofts Superset
für ein Typ Sicheres JS



Node.js

Serverseitige
JavaScript Plattform

Grundlagen



NPM

Paketmanager für
JavaScript



OpenAPI

Schnittstellendef. Von
RESTful APIs



Agile

Methode für flexibles
Projektmanagement



Wasserfall Modell

Traditioneller
sequenzieller Ansatz

Problemanalyse

45%

Budgetüberschreitung

45% der IT-Projekte haben Budgetüberschreitungen

7%

Zeitüberschreitung

7% der IT-Projekte haben Zeitüberschreitungen

Wirtschaftliche Faktoren

56%

Leistungsabfall

56% weniger wert als erwartet

30%

Projekt Erfolgsrate

30% der Projekte erreichen die gewünschten Ergebnisse





Problemanalyse

Code Qualität

- Wenige Entwickler schreiben Clean Code (direkt)
- Refactoring kostet Zeit
- Erschwert Wartung
- Erhöhter Zeit- und Kostenaufwand

Technische Faktoren

Technical debt

- Schnell Lösungen → technical debt
- Überarbeitung kostet Zeit
- Vermindert Qualität, erhöht Fehler
- Verlangsamt Entwicklung



Problemanalyse

Kommunikation

Schwierigkeiten durch
Sprache, Kultur oder Art



Anforderungsstabilität

Anzahl der Änderungen der
Anforderungen



Turnover

Personaländerungen im
Team

Koordination

Herausforderung der
Abstimmung



Lösungskonzept

Projektkontext

Codegenerierung um
Effizient und Qualität zu
verbessern

Impl. Codegenerierung

Standardisierung und
Beschleunigung der
Entwicklung

Design

Design des Konzepts und
Prozesses

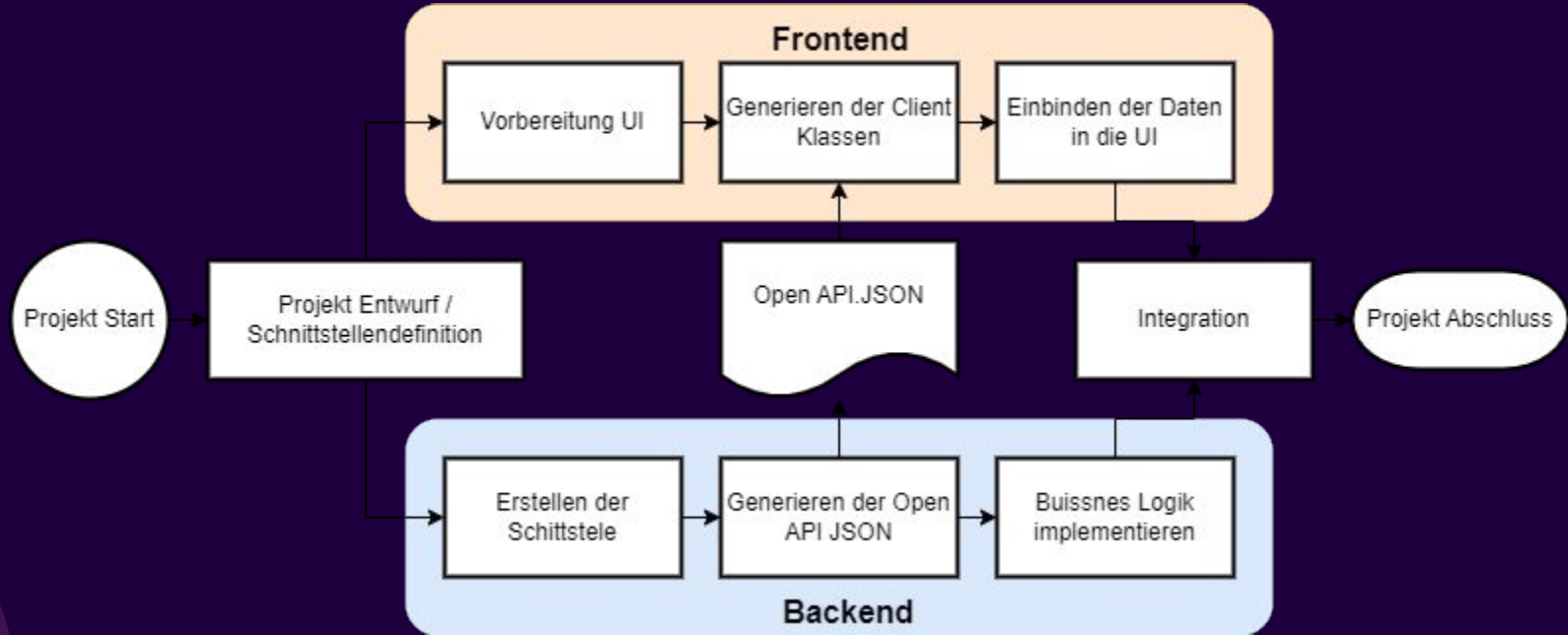
Teamkommunikation

Klare Prozessdefinierung
und Reduktion der
Kommunikation

Analyse

Bewertung der
Finanziellen und
operationellen Vorteile

Lösungskonzept

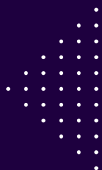




Umsetzung Lösungskonzept

Implementierung

Generierung von Angular Services	<code>generateAngularServices</code> analysiert OpenAPI-Spezifikation und generiert Angular Services entsprechend der definierten Endpunkte
Erstellung von Interfaces	Generierung von Interfaces Basierend auf Schemata durch <code>generatesInterfacesFromSchemas</code>
Dynamische Service-Generierung pro Tag	Zuordnung von API-Endpunkten zu Tags durch <code>generateServiceForTag</code>
Automatische Fehlerbehandlung	<code>generateMiddleWare</code> generiert Middleware zur Fehlerbehandlung
Automatische Methodengenerierung	<code>generateServiceMethod</code> erstellt Angular Services durch analyse von Api Pfaden



Umsetzung Lösungskonzept

Inbetriebnahme

Anpassung Config files

- Anpassung der `tsconfig.json` und `package.json` file
- Definition Speicherort der generierten Dateien
- Definition Eingangspunkt des Pakets
- Definition TypeScript Compiler und Speicherort kompilierter Dateien

Deployment

- Erstellung der `.npmrc` Datei
- Definition der Umgebungsvariablen und Zugangsdaten
- Build und Deployment
- Installierbar über
`npm install heigen@1.0.3`
und ausführbar über `npx heigen`



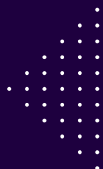


Abschluss

Evaluation

Def. und Implementierung eines Optimierten Workflows	Der Workflow wurde optimiert definiert. Wichtige Teile des Codes können nach API-Design generiert werden. Jedoch für einen spezifischen Anwendungsfall
Reduktion Boilerplate-Code	Der Einsatz von Codegenerierung führte zu einer Reduktion von zu schreibendem Boilerplate-Code. Komplexität einiger Anwendungen erfordert manuelle Anpassungen
Steigerung Effizienz, Qualität und Wartbarkeit	Durch den optimierten Workflow und die Automatisierung wurden die Entwicklungszeiten verkürzt. Jedoch für einen spezifischen Anwendungsfall
Sicherstellung der Erweiterbarkeit	Entwickelte Systeme bleiben trotz der Optimierungen und der Einführung automatisierter Prozesse flexibel und anpassbar. Allerdings Erweiterung potentiell notwendig.

In einer Firmeninternen nicht repräsentativen Umfrage bekam das Tool 4 Punkten für Zufriedenheit



Abschluss

Zusammenfassung & Ausblick

Zusammenfassung

- Erfolgreiche Integration von Codegenerierung
- Reduzierung Boilerplate-Code
- Optimierte Entwicklungsprozesse
- Allgemeine Erhöhung der Produktivität

Ausblick

- Weitere Anpassungen und Optimierungen
- Ausweitung der Automatisierung
- Einsatz von LLMs und NLP
- Ausweitung auf andere Frameworks

LIVE DEMO



Danke für eure Aufmerksamkeit!

Fragen?